3.5、形态操作

[OpenCV: Morphological Transformations]
https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

形态学操作是图像处理中非常实用的一类技术，主要用于二值图像（黑白图像，前景通常是白色 255，后景是黑色 0），通过“形状”来处理图像，常用于去噪、分离物体、填充孔洞、提取轮廓等。

1. 基本概念

形态学操作需要两个输入：

- 原始图像（通常是二值化的）
- 结构元素（Kernel / Structuring Element）：一个小的矩阵（如 5x5 全 1），决定了操作的“形状”和强度。
  最基础的两个操作是：
- 腐蚀（Erosion）
- 膨胀（Dilation）
  其他高级操作都是在这两个基础上组合而成的。

2. 腐蚀（Erosion）

原理：就像土壤被侵蚀一样，它会“侵蚀”前景物体（白色区域）的边界。

- 核（kernel）在图像上滑动。
- 只有当核覆盖的所有像素都是 1（白色）时，中心像素才保留为 1，否则变为 0。
  效果：
- 前景物体边界变细、变小
- 去除细小的白色噪声

- 可以分离粘连在一起的物体

```
import cv2 as cv
import numpy as np

img = cv.imread('j.png', cv.IMREAD_GRAYSCALE)  # 读取灰度图（最好是二值图）

kernel = np.ones((5,5), np.uint8)               # 5x5 矩形核
erosion = cv.erode(img, kernel, iterations=1)   # iterations=腐蚀次数

cv.imshow('Erosion', erosion)
cv.waitKey(0)
```

参数说明：

- kernel：结构元素
- iterations：重复腐蚀几次（次数越多，腐蚀越强）

3. 膨胀（Dilation）

原理：腐蚀的反操作，白的区域膨胀。

- 只要核覆盖的区域中至少有一个像素是 1，中心像素就变为 1。
  效果：
- 前景物体边界变粗、变大
- 可以连接断裂的物体
- 常用于在腐蚀去噪后，恢复物体原来的大小

```
dilation = cv.dilate(img, kernel, iterations=1)
```

实用技巧：先腐蚀去噪 → 再膨胀恢复大小（这就是开运算的基础）。

4. 开运算（Opening）

定义：先腐蚀，再膨胀
opening = dilate(erode(img, kernel))作用：

- 去除小的白色噪声（小白点）
- 保持物体整体形状基本不变
- 常用于“去噪”同时不破坏主要物体

```
opening = cv.morphologyEx(img, cv.MORPH_OPEN, kernel)
```

5. 闭运算（Closing）

定义：先膨胀，再腐蚀
closing = erode(dilate(img, kernel))作用：

- 填充前景物体内部的小黑洞、小裂缝
- 连接狭窄的缺口

- 使物体更完整、连通

```
closing = cv.morphologyEx(img, cv.MORPH_CLOSE, kernel)
```

6. 其他高级操作（使用同一个函数 morphologyEx）

```
# 形态学梯度（Morphological Gradient）—— 突出物体轮廓
gradient = cv.morphologyEx(img, cv.MORPH_GRADIENT, kernel)

# 顶帽（Top Hat）—— 原图-开运算 = 提取比周围亮的细节（小亮斑）
tophat = cv.morphologyEx(img, cv.MORPH_TOPHAT, kernel)

# 黑帽（Black Hat）—— 闭运算-原图 = 提取比周围暗的细节（小黑洞）
blackhat = cv.morphologyEx(img, cv.MORPH_BLACKHAT, kernel)
```

7. 结构元素（Kernel）的创建方式

不要总是用 np.ones((5,5), np.uint8)，OpenCV 提供了更灵活的函数：

```
# 矩形核（最常用）
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5,5))

# 椭圆核（圆润）
kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, (5,5))

# 十字核
kernel = cv.getStructuringElement(cv.MORPH_CROSS, (5,5))
```

核越大，操作影响的范围越广；形状不同，效果也不同（比如椭圆核更平滑）。

8. 完整示例代码

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

img = cv.imread('your_image.png', 0)  # 替换成你的二值图像

kernel = np.ones((5,5), np.uint8)

# 各种操作
erosion = cv.erode(img, kernel, iterations=1)
dilation = cv.dilate(img, kernel, iterations=1)
opening = cv.morphologyEx(img, cv.MORPH_OPEN, kernel)
closing = cv.morphologyEx(img, cv.MORPH_CLOSE, kernel)
gradient = cv.morphologyEx(img, cv.MORPH_GRADIENT, kernel)

# 显示（用 matplotlib 方便对比）
titles = ['Original', 'Erosion', 'Dilation', 'Opening', 'Closing', 'Gradient']
images = [img, erosion, dilation, opening, closing, gradient]

for i in range(6):
    plt.subplot(2, 3, i+1), plt.imshow(images[i], 'gray')
    plt.title(titles[i])
    plt.xticks([]), plt.yticks([])

plt.show()
```

9. 实用建议

- 预处理：形态学操作通常在二值图像上效果最好。先用阈值化（cv.threshold）或 Otsu 方法转成二值图。
- 核的大小：从 3x3 开始测试，逐步增大看效果。

- 迭代次数：iterations=2 或更多可以加强效果，但不要过度。
- 应用场景：
  - 去除噪声 → 开运算
  - 填充孔洞 → 闭运算
  - 提取边缘 → 形态学梯度
  - 文本处理、医学图像、工业检测等都很常用。